SIMD Advantage Profiling on Apple M5 Max (NEON), single thread

ECSE 4320/6320, Class-wide Project 1. Scalar and NEON builds of SAXPY, multiply, dot product and a 3-point stencil (f32, f64, u32) are compared across working sets, alignments, strides, data types and a roofline, with compiler evidence for every vectorization claim. All numbers come from this repository. Evidence for each figure is listed in its caption and in the Evidence index (last section). The study uses one thread, no hardware counters and no core pinning; limitations are listed in Section 11.

1. Summary

1. The SIMD speedup of f32 SAXPY depends on the scalar baseline more than on the lane count: 5.2x in L1 and 3.0x in DRAM over the default scalar build, but 2.1x and 1.35x over an unrolled scalar loop (Table 2).
2. One thread sustains 147 to 154 GiB/s of SAXPY traffic at every size from 256 KiB to 1 GiB while measured load latency rises from 0.7 ns (L1) through 2.6 and 7.1 ns to 111 ns (DRAM). The flat plateau is a core-side limit, not a latency or DRAM limit (Figure 4).
3. fp64 SAXPY gains only 1.07x over unrolled scalar in L1 and 1.02x in DRAM, strides of 2 or more gain nothing (within 10%), and the random gather gains at most 16%.
4. The f32 dot-product gain of fast-math over unrolled scalar in L1 (13.6x in the sweep, 14.0x when remeasured) splits into 8.4x from 16 independent scalar accumulators and 1.7x from SIMD. For f64 the SIMD part is 0.8x to 1.0x (Table 4).
5. Only 16 B alignment of y matters much: SAXPY is 26% slower with y alone offset by 4 B, 6% with x alone, 45% with both (32 KiB), and 22% with both at 256 MiB (Figure 6).
6. All four kernels are left of the ridge (0.92 FLOP/B; peak 146 GFLOP/s, memory ceiling 147 GiB/s) and none is compute bound; the strict f32 dot product is latency bound at 3% of peak (Figure 11).

2. Pre-experiment hypotheses

Expectations were committed before the corresponding data existed; outcomes were filled in afterwards. H1 to H20 were committed on 2026-10-06 (commit 027cb3e) before any kernel was timed; H11b, H11c and H21 in 394887d, H10b and H12b to H12e in 06a00fe, H13b to H21b in 91c7858 (all before their experiments); HA1 to HE2 on 2026-10-07 in 032af85 and HA6 in 9733d53, before the measurements they predict. The hypothesis text of every row is unchanged since the commit in which it first appeared (compared programmatically); only the outcome cells were edited later. Rows H1 to H20 were written before, and the added rows after, a short smoke run whose facts are disclosed in `docs/HYPOTHESES.md`. Full wording, reasons and measured values are in that file. "Partly supported" means the trend was right but the value or scope was not.

Table 1. Hypotheses, written before the data (`docs/HYPOTHESES.md`).

ID	Experiment	Expected trend	Reason	Outcome
H1	SIMD vs scalar, L1	f32 2.5x to 4x, f64 1.5x to 2x	scalar loop already unrolled; ports and loop overhead cap it	refuted
H2	SIMD vs scalar, DRAM	gain falls to 1.0x to 1.3x	both wait on memory	partly supported
H3	gain vs size	compression starts at the L2 boundary, gone by 4x SLC	finite per-core L2 bandwidth	refuted
H4	f32 dot, `-O3`	not vectorized (about 1.0x); fast-math 2x to 4x	reassociation changes rounding	refuted
H5	dot accuracy	fast-math error 1e-7 to 1e-6 at N = 1e6; strict grows like sqrt(N) eps	lane splitting shortens the chain	partly supported
H6	multiply vs SAXPY	same speedup curve, lower AI	same bytes, half the flops	partly supported
H7	stencil	vectorizes; speedup close to SAXPY in cache	cheap unaligned vector loads	partly supported
H8	f64 vs f32	f64 speedup about half in cache; equal GiB/s in DRAM	lane count, bytes	partly supported
H9	u32	speeds up like f32 in cache	same lanes and bytes	supported
H10	alignment	offsets 0 to 64 B change SAXPY by under 5% (L1, L2)	hardware handles misaligned accesses	refuted
H11	tail	N < 64 costs 3x to 10x more per element	fixed entry and epilogue cost	partly supported
H12	stride, gather	useful GiB/s 1/2, 1/4 at strides 2, 4; 1/8 or less at stride 8 and gather; SIMD about 1.0x	whole 128 B lines fetched; no NEON gather	mostly supported (stride 32 deviates, Section 8)
H13	single-thread triad	60 to 120 GB/s, the roofline ceiling	one core's outstanding misses	refuted (131 GB/s)
H14	FMA benchmark	latency 3 to 4 cycles; 12 to 16 accumulators; 130 to 150 GFLOP/s	Little's law	partly supported
H15	roofline	SAXPY and dot far left of the ridge; L1 limited by load/store	AI far below peak/BW	supported
H16	reduction, no fast-math	"cannot reorder" remark; scalar speed; fast-math 2x to 4x	see H4	refuted (mechanism); conclusion holds
H17	SAXPY beyond SLC	1.0x to 1.3x despite perfect vectorization	see H2	partly supported
H18	naive benchmark	unphysical numbers; corrected differs over 3x in at least one case	dead-code elimination, reuse	partly supported (N2 did not reproduce)
H19	noise	median CV under 3% (cache), 8% (DRAM)	QoS bias to a Super core	partly supported
H20	`native` build	within 5% of `auto`	NEON and FMA already default	supported
H11b	tail mod 16	sawtooth in N mod 16 below N of about 256	epilogue iterations	refuted
H11c	tail intercept	affine in N, intercept 5 to 15 ns	entry checks	refuted (2.2 ns)
H21	no `restrict`	within 10% at large N; slower below N = 64; y = x + 1 at scalar speed	runtime overlap check	partly supported
H10b	alignment	at most 5% (L1, L2) and 3% (DRAM)	see H10	refuted

ID	Experiment	Expected trend	Reason	Outcome
H12b to H12e	stride kernels	`auto` within 10% of unrolled scalar; forced vectorization not faster; growth to stride 32; gather 5x slower in DRAM	no cheap vector form	partly supported (all four)
H13b	triad scaling	110 to 160 GiB/s; 12 processes several times one	single core is the limit	supported
H14b	FMA peak	140 to 150 GFLOP/s fp32; 80% to 90% sustained	4 pipes at 4.58 GHz	supported
H15b	roofline	memory bound in DRAM; SAXPY, mul under 20% of peak in L1; only fast-math dot over 25%	load/store limit	partly supported
H16b	dot accuracy	strict 30 to 1000 eps (N = 1e3 to 1e6); fast-math under 10 eps	summation order	partly supported
H21b	overlap by 1 element	at least 2x slower; 4 to 6 cycles per element	serial dependence	partly supported
HA1 to HA3	pointer chase	L1 3 to 4 cycles to 128 KiB; L2 12 to 20 cycles to 16 MiB; DRAM 90 to 140 ns	cache hierarchy	supported, partly supported, supported
HA4	pointer chase	distinct SLC step, 25 to 70 ns, 16 to 256 MiB	SLC (low confidence)	refuted
HA5	chase vs bandwidth	lines in flight grow over 30x from L2 to DRAM	Little's law	partly supported
HA6	page-granular chase	rise above 2x at 48 to 80 pages	the 1 MiB step is a TLB effect	refuted
HB1, HB2	traffic accounting	out-of-place SAXPY within 5% of triad, 0.75 to 0.85 of in-place; ranking read > in-place > add, triad > copy, scale	write-allocate reads	supported (mechanism not isolated), refuted
HC1, HC2	dot decomposition	1 accumulator 3.1 cycles, 16 accumulators 0.5 to 0.7; chain break 4x to 6x, SIMD 2.3x to 3.5x	FMA latency, load ports	partly supported, refuted
HD1, HD2	relative misalignment	penalty per misaligned stream, not relative offset; x only < y only < both	split 16 B accesses	supported, supported
HE1, HE2	stride-32 jump	stride 24 low, stride 48 jumps like 32; independent of footprint page alignment	one line per access	refuted, supported

3. Setup and method

Apple M5 Max, macOS 26.5.2, Apple clang 21.0.0; 6 "Super" and 12 "Performance" cores, no SMT (logical equals physical core count in both tiers); 128 B cache lines, 16 KiB pages; Super tier L1D 128 KiB and L2 16 MiB shared by 6 cores (`sysctl`; the SLC size is not exposed). NEON is 128-bit: 4 x fp32, 2 x fp64, 4 x u32. SME is present but not used (streaming mode, not a drop-in SIMD path).

Kernels are instantiated one translation unit per kernel, data type and build, and called through a function pointer so that nothing is inlined or hoisted; results are sunk behind compiler barriers. Builds: `scalar` (`-O3 -fno-vectorize -fno-slp-vectorize`), `scalar_unroll8` (same plus an unroll-8 pragma; the primary baseline because the default scalar loop runs at one iteration per cycle), `auto` (`-O3`), `native` (`-mcpu=native`, resolves to `apple-m4`), `fastmath` (`-O3 -ffast-math`), and `*_norestrict`, `auto_forcevec` for the negative cases. `-ffp-contract=on` is used everywhere except fast-math.

Timing: 24 MHz counter (41.7 ns tick), at least 1 ms per sample, call overhead of 0.7 to 0.9 ns subtracted, median with IQR error bars, thermal pressure sampled before and after every sample (no sample in any run was throttled). The harness thread runs at QoS user-interactive; macOS cannot pin threads.

item	setting
Repetitions	15 samples per point (3 passes x 5 repetitions, configuration order reshuffled each pass) for sweeps, alignment, tail, stride, traffic, dot decomposition, misalignment, stride-jump and both pointer-chase runs; 15 repetitions per accumulator count for the FMA peak; 10 per process for the multi-process triad; 5 runs per naive-benchmark variant; 100 per session for the clock estimate
Data initialization	fixed seed (12345 combined with N), uniform values in [0.5, 1.5) for f32 and f64 (no denormals), random odd integers for u32; arrays are first-touched during initialization, outside timing; the in-place operand of SAXPY is re-initialized before every sample; SAXPY uses a = 1e-6 so that y stays near 1
Frequency policy	no frequency control is available; the effective clock is estimated each session from a dependent add chain and cycle counts assume it is constant; AC power, Low Power Mode off, `caffeinate` around every run
Machine state	load average 1.2 to 2.4 in the earlier sessions; 2.8 to 3.4 in the final session (a video player was running; Parallels and a torrent client were closed beforehand)

Effective clock: 4.58 GHz (IQR half-width 0.015 GHz) in the main sessions and 4.55 GHz (half-width 0.037 GHz) in the session of the pointer-chase, traffic, dot-decomposition, misalignment and stride-jump runs; cycle counts use the clock of their own session. A dependent integer multiply chain gives 3.00 cycles at 4.58 GHz, supporting the 1-cycle-add assumption. Powermetrics samples taken afterwards show the Performance clusters topping out at 4380 MHz and the Super cluster at 4608 MHz, and the Super cluster fully active at 4608 MHz per busy core during a SAXPY run (cluster averages 4548 to 4596 MHz). The benchmark therefore ran on Super cores, the thread migrated among the six of them, and cycles per element carry about 1% uncertainty. Only one benchmark (f32 SAXPY at 32 MiB) was sampled this way. The L1 cliff at 128 KiB, not 64 KiB, agrees.

Correctness: every kernel, data type and build over 29 sizes and 7 start offsets with guard regions, against a double-double reference (`long double` is only 64 bits on this ABI): at most 1 ulp for SAXPY, multiply, triad, strided and gather (observed 0), 2 ulp for the stencil (observed 1), exact for u32, dot error at most N x eps. The kernels added later (memory-traffic kernels and dot with 1, 4, 8, 16 accumulators) have their own suite. A mutation check shows that the suite fails for an off-by-one loop bound and a wrong stencil weight. All 80562 checks pass at the final commit.

4. Vectorization verification

Compiler remarks and loop bodies exist for every kernel, data type and build. Facts that follow from them:

- SAXPY, multiply, stencil, u32 dot and fast-math dot are vectorized 4 x 4 (f32, u32) or 2 x 4 (f64): 16 (8) elements per iteration, four independent `fmla` and paired `ldp q` / `stp q`, then a 4-element vector loop and a scalar loop. NEON has no masking, so tails use these

extra loops.

- Scalar builds report "loop not vectorized"; `scalar` is not unrolled (6 instructions per element), `scalar_unroll8` has 8 `fmadd` per 8 elements.
- The f32 dot product at plain `-O3` is vectorized (width 4, interleave 4), but the loop body is `fmul.4s` followed by 16 serial scalar `fadd`: a strict in-order float reduction cannot be reassociated, so the compiler keeps a serial add chain even when the multiplies are vectorized. The u32 dot uses four `mla.4s` accumulators without any flag because integer addition is associative.
- Strided SAXPY (stride 2 to 48) and gather are not vectorized by `auto` (the cost model reports vectorization as not beneficial; the scalar loop is interleaved 4x). Forced vectorization yields `ld2.4s` at stride 2, 2-lane `zip1` at stride 8 and lane inserts for the gather, always with lane stores.
- `restrict` on local pointers had no effect; it must sit on function parameters. The kernels were fixed accordingly and `auto_norestrict` is kept as the negative case. Data from the early smoke runs predate this fix and are not used in any plot.

Evidence for SAXPY f32, `auto` (disassembly excerpt, `x1` = x, `x2` = y, `v0[0]` = a):

```
38: ldp  q1, q2, [x9, #-0x20]   ; load 8 floats of x  (ldp = 2 x 128-bit loads)
3c: ldp  q3, q4, [x9], #0x40    ; 8 more of x, advance 64 B  -> 16 floats of x
40: ldp  q5, q6, [x10, #-0x20]  ; 8 floats of y
44: ldp  q7, q16, [x10]         ; 8 more of y                -> 16 floats of y
48: fmla.4s v5,  v1, v0[0]      ; y += a*x on 4 lanes (fused multiply-add)
4c: fmla.4s v6,  v2, v0[0]      ; 4 independent fmla = interleave count 4, 16 elements per iteration
50: fmla.4s v7,  v3, v0[0]
54: fmla.4s v16, v4, v0[0]
58: stp  q5, q6, [x10, #-0x20]  ; store 8 results
5c: stp  q7, q16, [x10], #0x40  ; store 8 results, advance
60: subs x11, x11, #0x10        ; 16 elements per trip; loop ends on b.ne (no mask, no predicate)
90: ldr q1,[x11],#0x10 ... fmla.4s v2,v1,v0[0] ... str q2,[x10],#0x10   ; 4-element vector loop (N mod 16 >= 4)
c0: ldr s1,[x9],#4 ... fmadd s1,s0,s1,s2 ... str s1,[x10],#4          ; scalar epilogue (N mod 4 elements)
```

Remark for the same loop: `vectorized loop (vectorization width: 4, interleaved count: 4)`. Remark for strided SAXPY (stride 2): `the cost-model indicates that vectorization is not beneficial`, then `interleaved loop (interleaved count: 4)`, which is scalar code.

5. Locality, SIMD gain and data types

Hypothesis. H1 to H3 and H6 to H9: gain is below the lane count, compresses toward 1x beyond L2, fp64 gains about half of fp32.

Result. Speedup of `auto` over `scalar_unroll8` (over `scalar` in parentheses), median ratio:

kernel	f32 L1 (32 KiB)	f32 L2 (4 MiB)	f32 DRAM (1 GiB)	f64 L1	f64 DRAM	f64 GiB/s L1 / DRAM
SAXPY	2.08 (5.18)	1.37 (3.05)	1.35 (3.03)	1.07 (2.63)	1.02 (1.55)	257 / 146
multiply	2.20 (5.17)	1.07 (2.39)	1.17 (2.51)	1.07 (2.52)	1.04 (1.29)	251 / 121
stencil	2.92 (3.77)	2.57 (3.37)	2.54 (3.33)	1.47 (1.95)	1.38 (1.83)	127 / 115
dot, strict `-O3`	1.46 (1.63)	1.52 (1.59)	1.48 (1.59)	1.39 (1.56)	1.40 (1.51)	33 / 30
dot, `-ffast-math`	13.6	11.5	10.1	6.7	5.0	

Table 2. Speedup of the vector build (`data/processed/exp1_key_table_phase3.csv`).

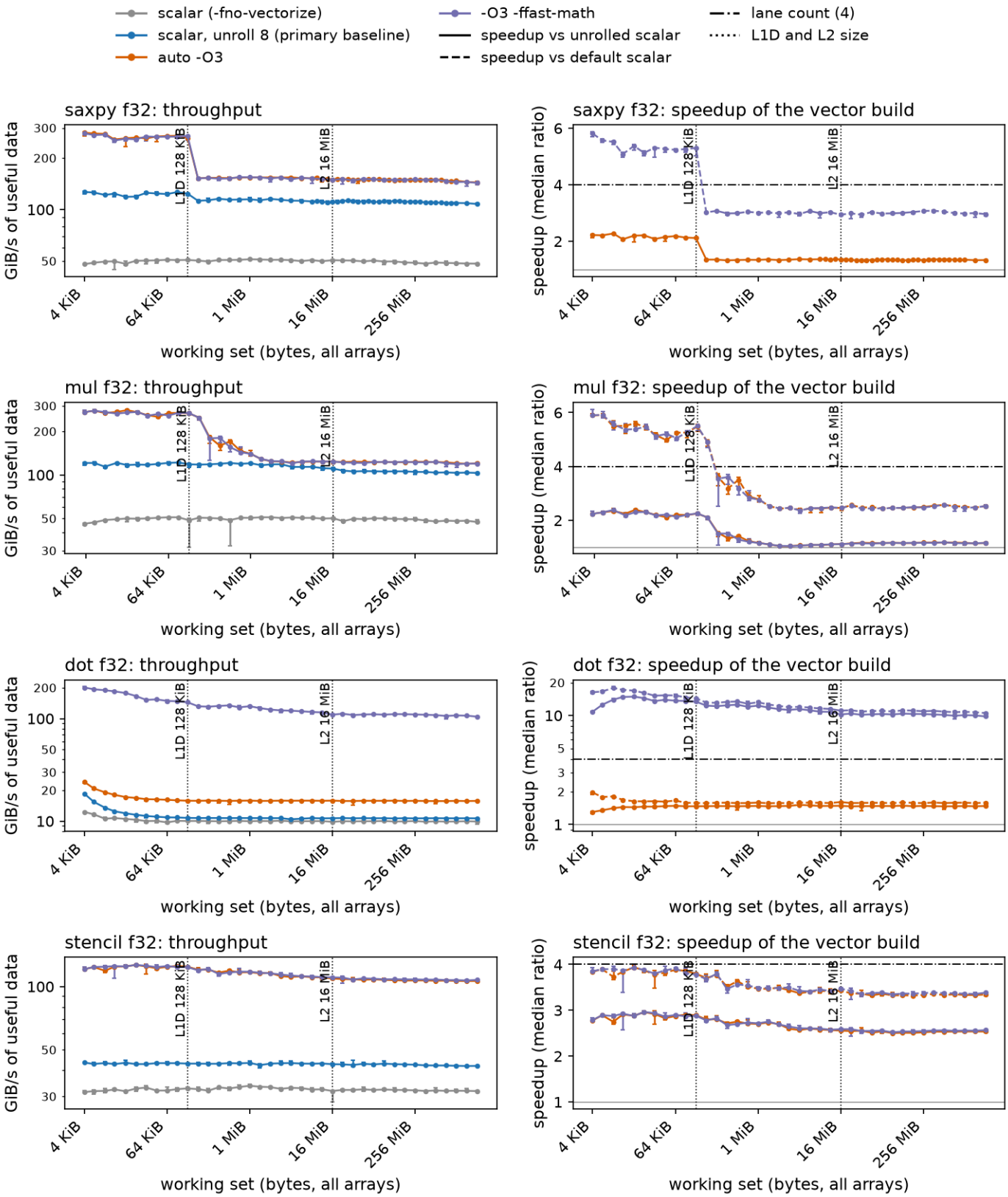


Figure 1. f32, one row per kernel: throughput (left) and speedup over the unrolled and the default scalar build (right) versus working set; look for the single step at the 128 KiB L1 boundary and the absence of any feature at 16 MiB (`plots/exp1_sweep_f32.png`, `data/processed/exp1_speedup_phase3.csv`, raw `data/raw/sweep_phase3_*.csv`). Bars are IQR.

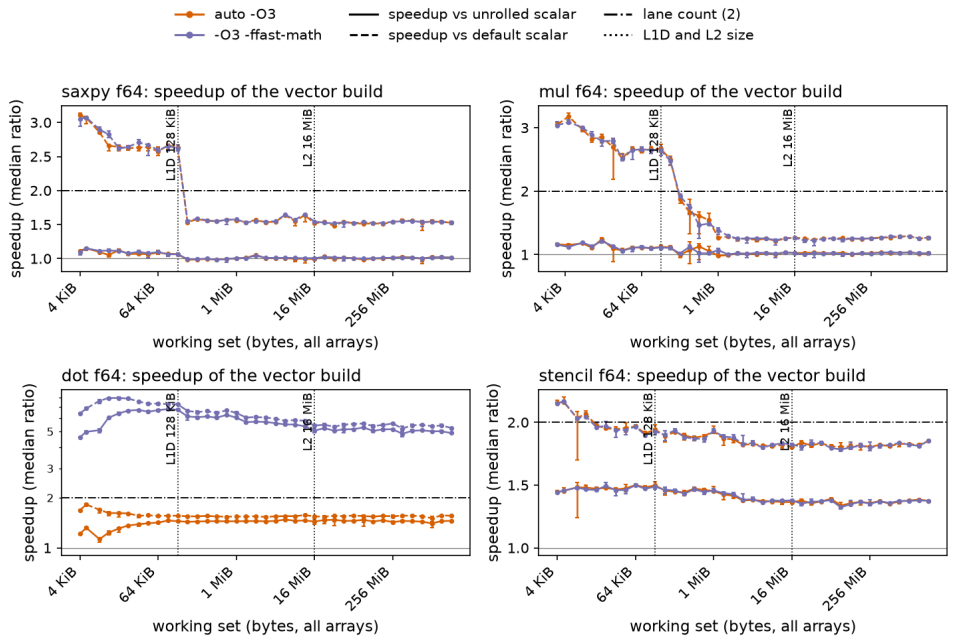


Figure 2. f64 speedups, same layout; the dotted-dashed line is the lane count (2) (`plots/exp1_sweep_f64.png`).

f32 SAXPY runs at 260 GiB/s (0.197 cycles per element) in L1 and at 147 to 154 GiB/s (0.33 to 0.35 cycles) from 256 KiB to 1 GiB, declining slowly to 143 GiB/s at 2 GiB (Figure 3). Multiply (three arrays) declines gradually between 128 KiB and 2 MiB. `native` equals `auto` within 3%. Median CV is 1.8% in cache and 1.6% in DRAM (90th percentile 5%, a few points up to 47%).

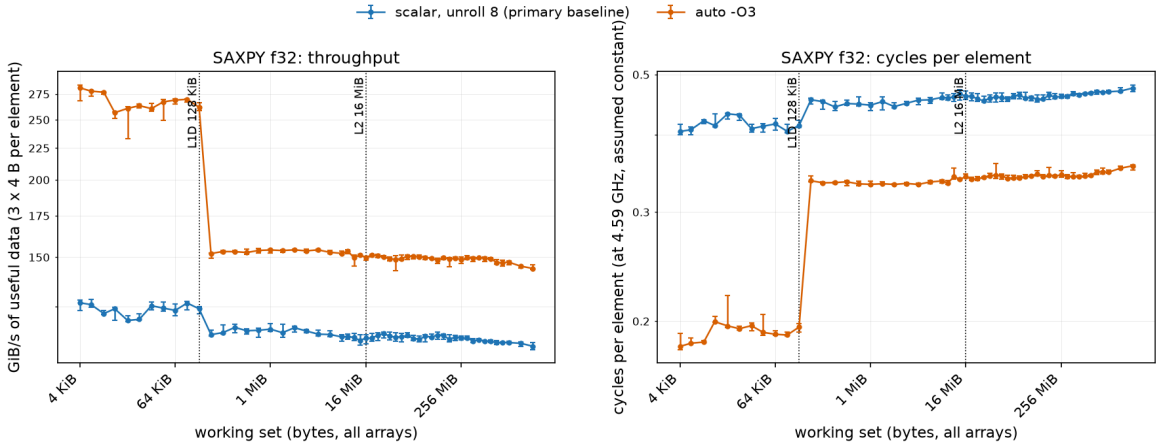


Figure 3. f32 SAXPY locality sweep: GiB/s (left) and cycles per element (right) from 4 KiB to 2 GiB; look for one step at 128 KiB and a flat plateau afterwards, with L1D and L2 sizes marked (`plots/exp2_saxpy_locality.png`, raw `data/raw/sweep_phase3_D.csv`).

Pointer-chase annotation of the locality curve. A random single-cycle pointer chase (one dependent load per step, one 128 B node per line, the permutation verified to be one full cycle before timing) gives load-to-use latency per level (Figure 4a):

region	latency	cycles (4.55 GHz)
4 to 92 KiB	0.66 ns	3.0
152 to 190 KiB	1.95 to 2.3 ns	9 to 10 (transition)
256 KiB to 0.7 MiB	2.6 ns	12
1.25 to 11 MiB	7.1 to 9.4 ns	32 to 43
13.5 to 54 MiB	18.8 to 94 ns, rising smoothly	86 to 430
76 MiB and above	100 to 106 ns (111 ns at 512 MiB to 1.7 GiB)	455 to 504

Table 3. Pointer-chase latency by region (`data/processed/extra_chase_ext1.csv`).

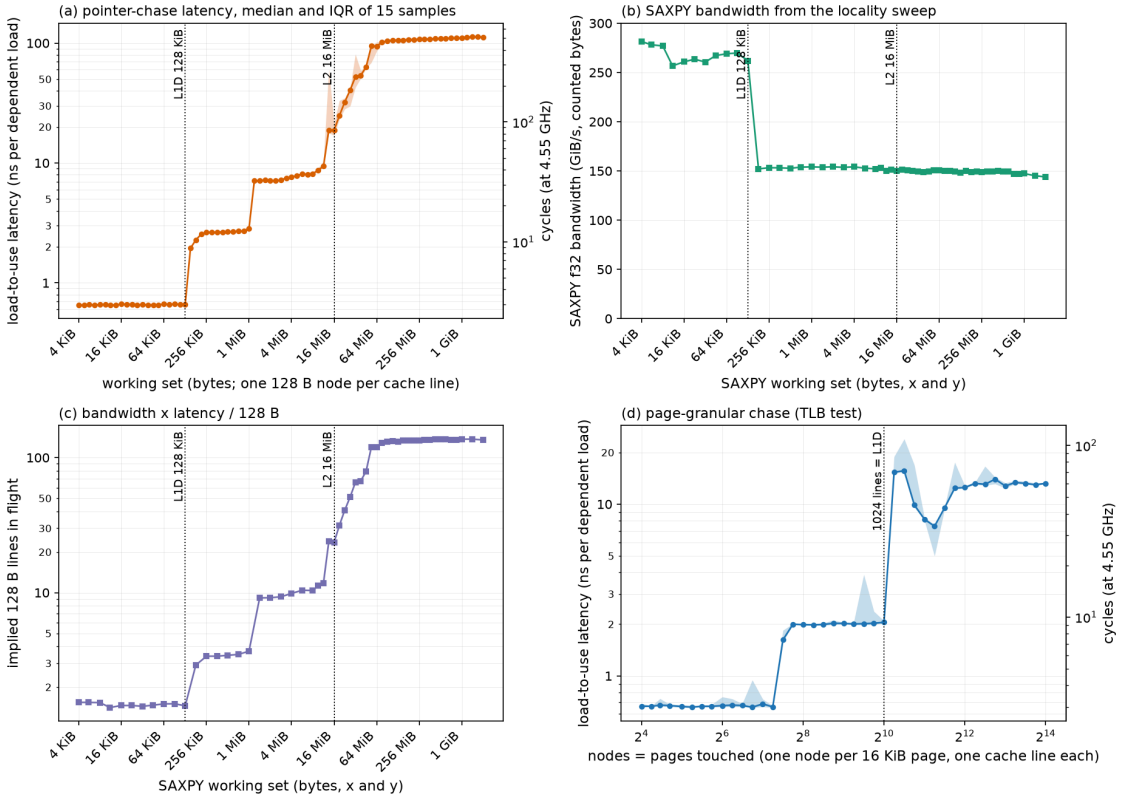


Figure 4. (a) pointer-chase latency with the cycle axis on the right, (b) the SAXPY bandwidth curve, (c) lines in flight implied by bandwidth x latency / 128 B, (d) page-granular chase (one node per 16 KiB page); look for steps at 128 KiB, about 1 MiB and about 13 MiB in (a) against the flat curve in (b) (`plots/exp9_chase_latency.png`, `data/processed/extra_chase_ext1.csv`, `data/processed/extra_chasepage_ext1.csv`, raw `data/raw/chase_ext1.csv`, `data/raw/chasepage_ext1.csv`).

Interpretation. The gain against `scalar_unroll8` compresses at the L1 boundary (2.1x to 1.35x for SAXPY), not at L2 or DRAM (H3 refuted). The bandwidth plateau is a real core-limited plateau: latency rises 14.6x between 4 MiB and 1 GiB while bandwidth falls 5% (154 to 147 GiB/s), so the lines in flight implied by bandwidth x latency grow from 9.9 to 137, and a fixed miss-queue limit cannot be the cause. A constant per-core transfer rate of about 35 counted bytes per cycle, fed by hardware prefetch, fits; with 12 concurrent processes the aggregate triad rate reaches 441 GiB/s (3.6x one process, Figure 12), so DRAM is not the limit. The cause inside the core (L2 fill or load/store throughput) was not tested. The chase shows three latency levels between L1 and DRAM and no distinct SLC plateau between 16 MiB and 76 MiB, only a smooth rise; no SLC size is claimed. Near 16 MiB the rise is consistent with the L2 capacity (13.5 MiB is the first point above 12 ns). The fp64 gain is half the f32 gain, and in cycles per element f64 vector and unrolled scalar are within 7% (0.40 vs 0.43), which points to a memory-operation limit (inference from cycle counts). u32 behaves like f32 (SAXPY 2.65x, multiply 2.30x, dot 5.9x in L1; `plots/exp5_dtype.png`, `plots/exp5_elements_per_ns.png`).

Failure or anomaly. (1) A latency step to 7.1 ns near 1 MiB matches no cache size reported by `sysctl`. A page-granular chase (Figure 4d) showed no latency rise at 48 to 80 pages (0.67 ns, equal to L1), so it is not L1 TLB reach (HA6 refuted); the first TLB-like rise occurs near 181 pages (about 2.0 ns). The cause of the 1 MiB step is unknown. (2) The dip near 16 to 32 KiB in the f32 SAXPY curve (about 255 against 275 GiB/s) is repeatable but unexplained. (3) Lines in flight are derived from counted bytes (no write-allocate), a stated convention (Section 9).

6. Dot-product decomposition

Hypothesis. HC1, HC2: a dependent chain costs about 3.1 cycles per element; 16 accumulators reach 0.5 to 0.7; breaking the chain explains 4x to 6x of the gain and SIMD the remaining 2.3x to 3.5x.

Result. Cycles per element (f32 first, f64 in parentheses), session with 4.55 GHz clock:

build	L1 (32 KiB)	L2 (4 MiB)	DRAM (1 GiB)
strict scalar, unroll 8 (baseline)	3.01 (2.91)	3.17 (3.20)	3.20 (3.26)
1 accumulator, scalar	3.39 (3.38)	3.39 (3.40)	3.44 (3.47)
4 accumulators, scalar	0.90 (0.88)	0.94 (0.94)	0.96 (0.97)
8 accumulators, scalar	0.46 (0.44)	0.48 (0.58)	0.49 (0.64)
16 accumulators, scalar	0.36 (0.37)	0.38 (0.58)	0.40 (0.64)
16 accumulators, NEON (auto)	0.23 (0.35)	0.29 (0.56)	0.32 (0.65)
plain loop, `-ffast-math`	0.21 (0.45)	0.30 (0.59)	0.32 (0.64)
chain-break gain (baseline / best scalar)	8.4x (7.9x)	8.2x (5.5x)	8.0x (5.1x)
SIMD gain (best scalar / fast-math)	1.67x (0.81x)	1.29x (0.97x)	1.25x (0.99x)
total (baseline / fast-math)	14.0x (6.4x)	10.6x (5.4x)	10.0x (5.1x)

Table 4. Dot-product decomposition (`data/processed/extra_dot_decomposition_ext1.csv`).

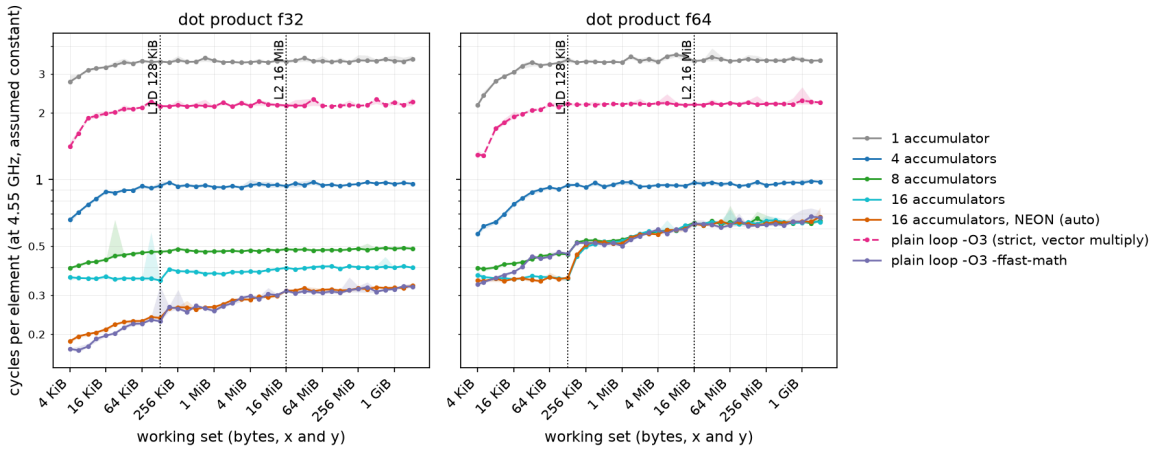


Figure 5. Cycles per element versus working set for 1, 4, 8, 16 scalar accumulators, the NEON build of the 16-accumulator code, and the plain loop with strict and fast-math summation; look for the separation between the chain-bound curves (3.4, 2.2) and the accumulator curves, and for the f64 curves merging at 0.6 beyond L1 (`plots/exp11_dot_decomposition.png`, raw `data/raw/dotdec_ext1.csv`).

Interpretation. Most of the 13.6x to 14.0x gain comes from independent accumulators, not from vector width. In f32 L1, 16 scalar accumulators already reach 0.36 cycles per element (8.4x), and NEON adds 1.7x. The scalar loop is faster than the two-loads-per-element count suggests because it uses `ldp s, s` (two elements per load op; disassembly of the scalar 16-accumulator object), so it is bound by load operations (about 3 per cycle), not by FMA latency. In f64 the 16-accumulator scalar loop equals the NEON build beyond L1, so SIMD gives nothing there (inference: both are bound by loads of 8 B elements). The strict f32 dot product (2.1 to 2.2 cycles per element) is the serial `fadd` chain behind vectorized multiplies.

Failure or anomaly. The 1-accumulator scalar loop (3.39) is 9% slower than the unrolled strict loop (3.01) in f32 L1, likely loop overhead on the critical path (not tested). The earlier sweep gave 13.6x and this session 14.0x for the same ratio (different sessions and clock estimates).

7. Alignment and tail

Hypothesis. H10, H10b, HD1, HD2: alignment changes SAXPY by under 5% (H10, written first); relative misalignment depends on the stream, not on the relative offset (HD1).

Result. A first test shifted both arrays equally by 0 to 132 B: multiples of 16 B ran at the offset-0 speed (within 1%); all other offsets, including 4, 8 and 12 B, were 40% slower at 32 KiB, 24% at 4 MiB and 21% at 256 MiB; the unrolled scalar loop varied by at most 6%. The relative-offset test shifted x and y independently over 0, 4, 8, 16, 32, 64 B (Figure 6). Time relative to (0, 0), `auto` build, median of 15:

offsets (x, y)	32 KiB	256 MiB
x only, 4 B	1.06	1.07
y only, 4 B	1.26	1.16
both 4 B	1.45	1.22
x 4 B, y 8 B	1.39	1.21
x 16 B, y 4 B	1.29	1.19
x 4 B, y 16 B	1.01	1.04
x 64 B, y 4 B	1.28	1.18
multiples of 16 B on both	1.00 to 1.04	0.99 to 1.06

Table 5. Relative misalignment (`data/processed/extra_misalign_ext1.csv`); the unrolled scalar control stays within 5.5% in every cell.

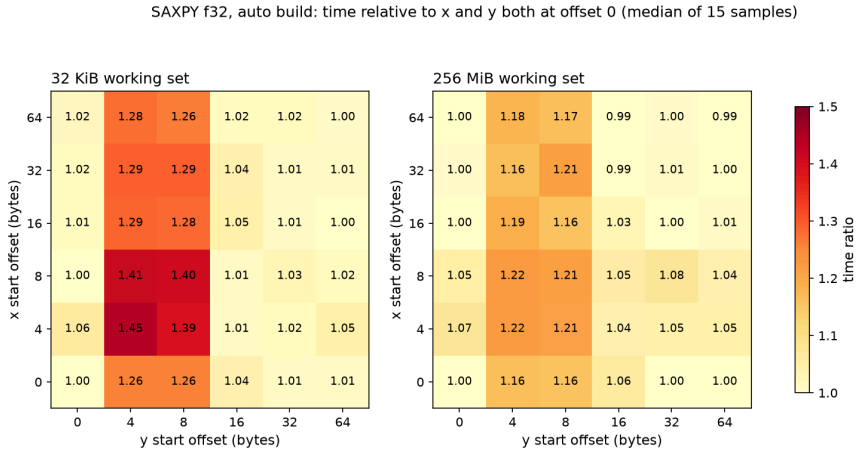


Figure 6. SAXPY f32 `auto`: time ratio against (0, 0) for x and y start offsets; look for the penalty in the y = 4 and y = 8 columns and its absence in the 16, 32, 64 columns and in the x only row (`plots/exp12_relative_misalignment.png`, raw `data/raw/misalign_ext1.csv`; equal-offset data in `plots/exp3_alignment.png`).

Tail (N up to 16): 0.4 to 1.2 cycles per element; 0.35 at N = 32, 0.25 at N = 64, 0.19 beyond N = 120; fixed cost about 2 ns (intercept of an affine fit over N = 32 to 160). There is no dependence on N mod 4 or mod 16 around N = 256, 1024 and 4096 above the roughly 1 ns noise.

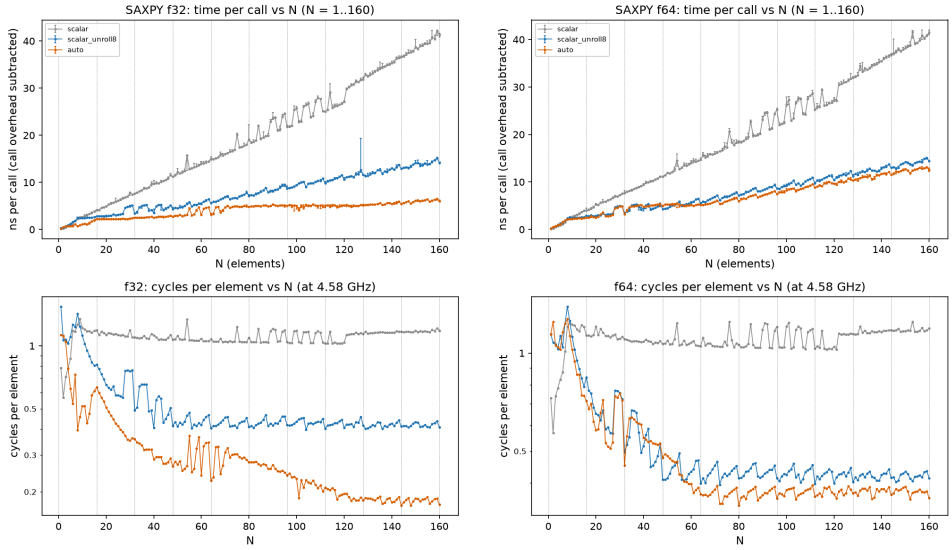


Figure 7. SAXPY time per call (top) and cycles per element (bottom) for N = 1 to 160, f32 and f64; look for the fall toward 0.19 cycles per element and the irregular curve below N = 64 (`plots/exp3_tail_small_n.png`; mod-16 view `plots/exp3_tail_mod16.png`; raw `data/raw/tail_phase4.csv`).

Interpretation. The slowdown follows the misalignment of y mainly (loaded and stored), the misalignment of x only secondarily, and does not depend on the relative offset (x 4, y 8 equals x 4, y 4 within 4%). A misaligned x costs little alone (6%) but 19 points on top of a misaligned y, so the effects are not additive. Offsets of 16 B or more behave like 0 B, including a 64 B shift of x only. What the data explain: the penalty is a function of each stream's offset modulo 16 B. What they do not explain: the mechanism (a 16 B access crossing a 16 B boundary being split is a guess, no counters), and why x alone costs so little.

Failure or anomaly. The disassembly predicts three code paths (N < 4, N < 16, N >= 16); no clear steps appear at these thresholds, and below N = 64 the curve is irregular without an explanation (H11b refuted).

8. Stride and gather

Hypothesis. H12 to H12e and HE1, HE2: strides 2 and 4 are scalar and fall to 1/2 and 1/4 of unit-stride useful bandwidth beyond L1; the SIMD gain is 1.0x; the stride-32 jump is a threshold at one line per access (HE1) independent of footprint alignment (HE2).

Result. f32 SAXPY with strides 1 to 48 and a random gather (useful bytes counted):

(f32)	stride 1	2	4	8	16	24	32	48	gather
512 MiB, ns per element	0.075	0.200	0.298	0.552	1.17	1.42	11.5	3.96	2.94
512 MiB, useful GiB/s	149	56	37.5	20.3	9.5	7.9	1.0	2.8	5.1
32 KiB, ns per element	0.042	0.167	0.166	0.171	0.227	n/m	0.375	n/m	0.136

Table 6. Strided access; n/m = not measured; strides 24 and 48 from the later session (`data/processed/phase4_summary_phase4.txt`, `data/processed/extra_summary_ext1.txt`).

`auto` equals the unrolled scalar loop at every stride of 2 or more within 10%; forcing vectorization is not faster, 74% slower at stride 4 in L1 (lane stores) and 11% faster at 512 MiB stride 2. The gather is 39x slower per element than unit stride in DRAM (2.9 vs 0.075 ns).

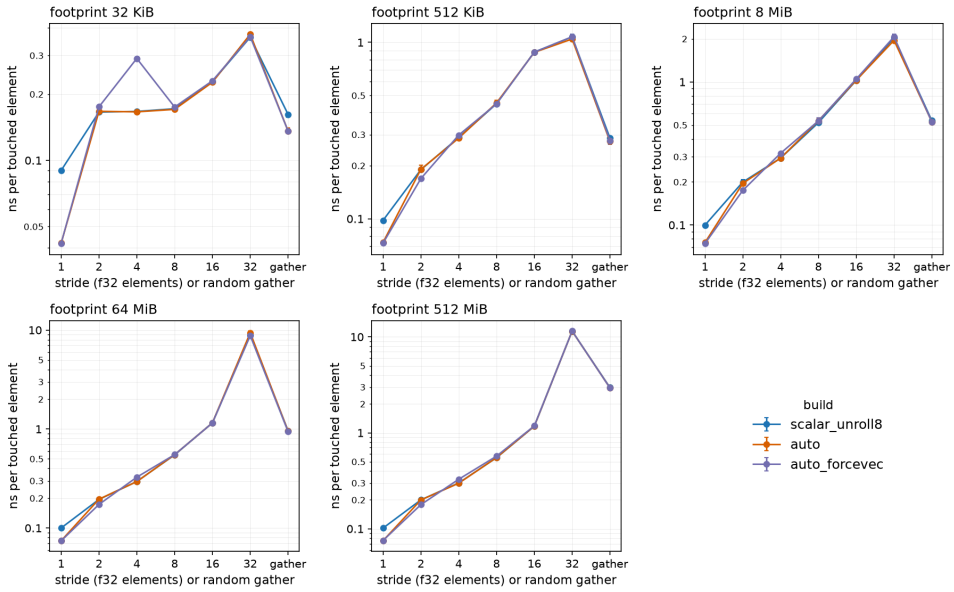


Figure 8. f32 SAXPY with strides 1 to 32 and a random gather, ns per touched element at five footprints; look for the monotonic growth with stride and the peak at stride 32 in the three largest footprints (`plots/exp4_stride_gather.png`, raw `data/raw/stride_phase4.csv`).

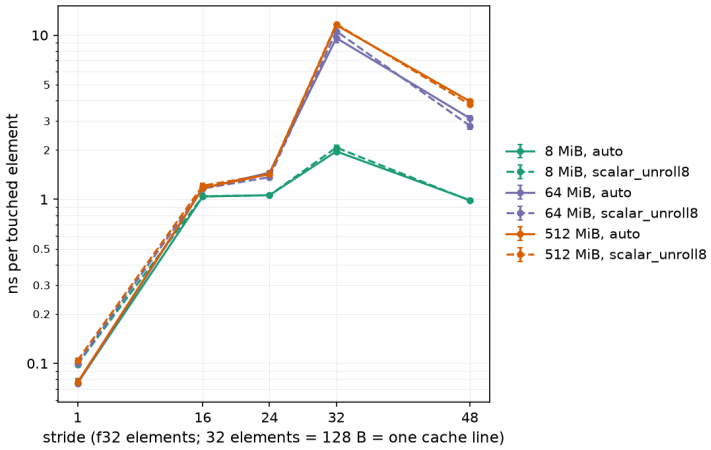


Figure 9. Stride-jump probe at 8 MiB, 64 MiB and 512 MiB, strides 1, 16, 24, 32, 48; look for the peak at 32 and the partial recovery at 48 (`plots/exp13_stride_jump.png`, raw `data/raw/stridejump_ext1.csv`).

Interpretation. Strides 2 to 48 give no SIMD gain because neither the compiler nor forced vectorization has a cheap vector form (no NEON gather or scatter). Time per element grows with the bytes of line traffic per element up to stride 16 (0.075 to 1.17 ns). The stride-32 spike is not explained by a one-line-per-access threshold: stride 48 (192 B, also one new line per access) costs 3.4x of stride 16, not more than 5x, and stride 24 costs 1.2x of stride 16. A page-multiple footprint is not the cause: n = 2097152, 2097153 and 2097216 give 11.52, 11.68 and 11.58 ns per element (within 1.4%).

Failure or anomaly. From stride 16 (64 B) to stride 32 (128 B) the time per element grows 10x (1.17 to 11.5 ns at 512 MiB) although the line traffic only doubles, and it falls at stride 48. At 8 MiB (L2 resident) the same step is 1.9x, so the effect appears beyond L2. Cause unverified (a prefetcher or DRAM-mapping effect specific to a 128 B stride is a guess). Useful bandwidth at stride 32 is 0.0065 of unit stride, not the 1/32 = 0.031 a line-traffic model gives (H12's other predictions held).

9. Memory ceiling and roofline

Hypothesis. H13, H14, H15 and HB1, HB2: the one-thread triad is the memory ceiling; write-allocate reads make triad underreport; counted bandwidth ranks read-only highest.

Result. Counted traffic convention: compulsory reads plus writes in bytes (read 4, `incr` 8, copy 8, scale 8, add 12, triad 12, SAXPY 12, out-of-place SAXPY 12 per f32 element), without write-allocate fills. `auto`, f32, GiB/s of counted traffic:

footprint	read-only	incr	SAXPY in place	copy	scale	add	triad	SAXPY out of place
4 MiB	111.8	146.3	146.5	121.3	120.4	124.1	123.3	123.4
256 MiB	81.5	131.2	146.5	116.2	117.6	121.3	117.4	121.7
1 GiB	80.5	127.5	139.4	115.1	120.2	117.6	113.7	118.0

Table 7. Counted bandwidth by pattern (`data/processed/extra_traffic_ext1.csv`).

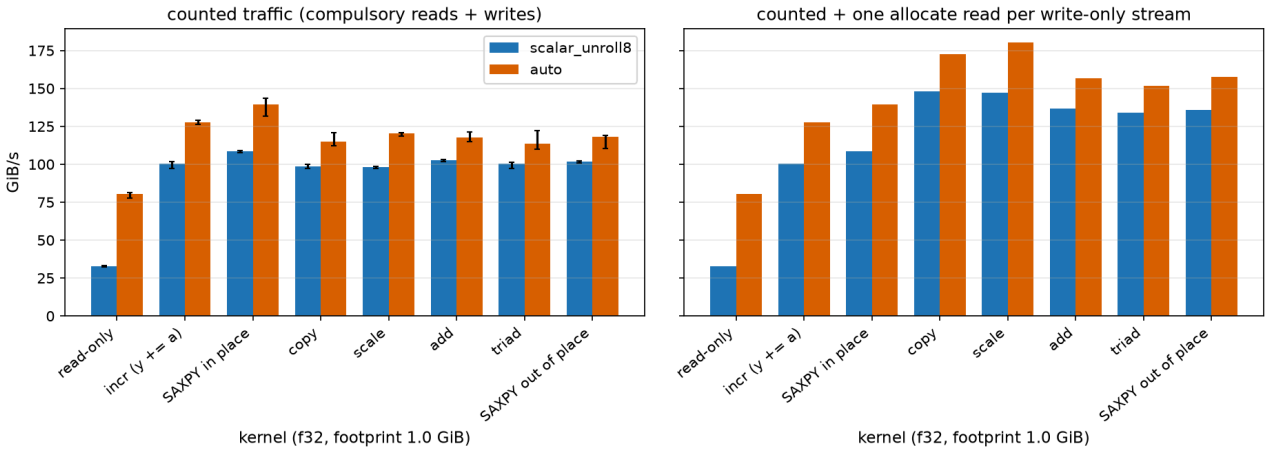


Figure 10. Counted bandwidth by access pattern at a 1 GiB footprint (left) and the same bytes with one extra allocate read per write-only stream (right); look for the unequal bars on the right (`plots/exp10_traffic_accounting.png`, raw `data/raw/ceiling_ext1.csv`).

Out-of-place SAXPY (three arrays, like triad) is within 0.1% to 3.8% of triad and 0.83 to 0.85 of in-place SAXPY at 4 MiB to 1 GiB. Adding one allocate read per write-only stream puts triad +9%, add +13% and out-of-place SAXPY +13% above in-place SAXPY at 1 GiB, but copy +24% and scale +29%. Read-only is the slowest pattern in DRAM (80 GiB/s). The roofline uses one memory ceiling, 147 GiB/s: the highest counted DRAM-size bandwidth of any pattern (in-place SAXPY; the SAXPY sweep at 1 GiB gives the same 147.3 GiB/s). Triad counted (122 GiB/s) and triad with an allocate read (162 GiB/s, unverified) are drawn as alternatives. Compute peak: 3.99 `fmla.4s` per cycle at 20 accumulators = 145.8 GFLOP/s fp32 (IQR 145.7 to 146.9), fp64 72.3 GFLOP/s; the plateau for most accumulator counts is 3.4 per cycle (85%); four FP pipes are inferred. Ridge: 0.92 FLOP/B (1.11 with the triad ceiling).

kernel (f32, `auto`)	AI (FLOP/B)	GFLOP/s L1 / DRAM	% of peak L1 / DRAM	% of memory ceiling, DRAM
SAXPY	0.167	46.6 / 26.4	32 / 18	100
multiply	0.083	23.1 / 10.8	16 / 7	82
dot, strict	0.25	4.4 / 4.2	3 / 3	11
dot, fast-math	0.25	40.8 / 28.8	28 / 20	73
stencil	0.625	83.7 / 71.4	57 / 49	72

Table 8. Roofline coordinates (`data/processed/roofline_points_phase5.csv`).

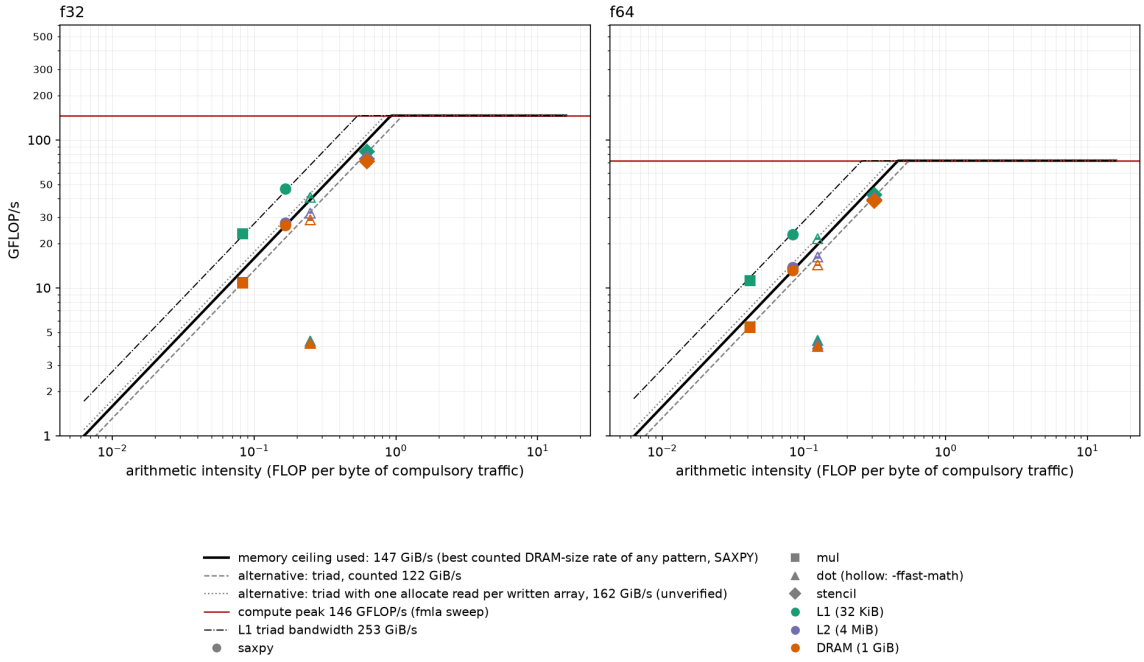


Figure 11. Single-thread roofline for f32 (left) and f64 (right); colours mark the working-set regime, hollow triangles the fast-math dot product; the solid line is the memory ceiling used, with the alternatives and the compute peak in the legend below (`plots/exp6_roofline.png`, `data/processed/roofline_points_phase5.csv`).

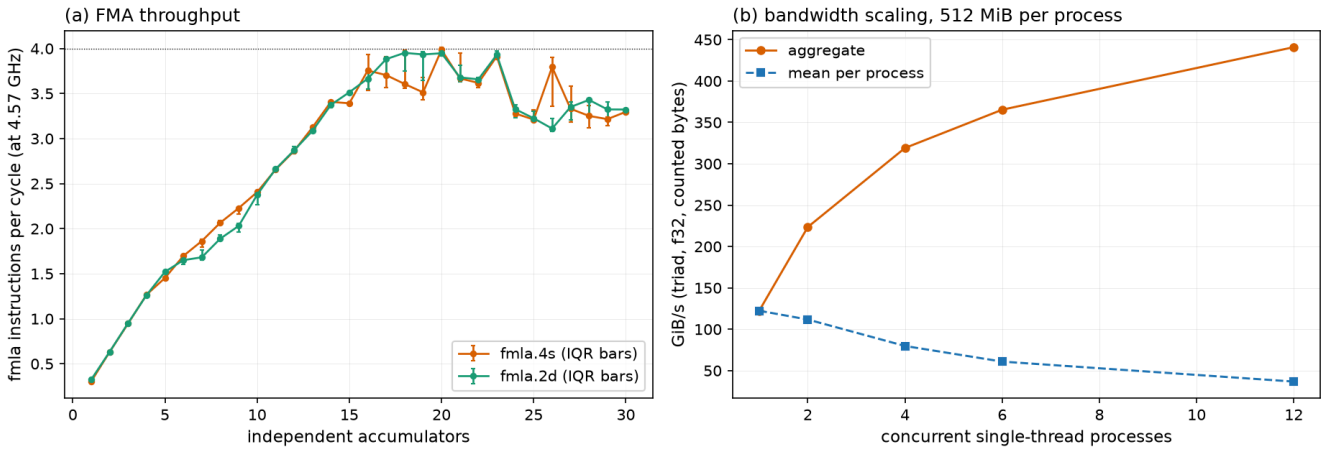


Figure 12. (a) FMA throughput versus independent accumulators; (b) triad bandwidth versus concurrent single-thread processes; look for the 4 per cycle maximum and the 441 GiB/s aggregate at 12 processes (`plots/exp6_peaks.png`, raw `data/raw/fmapeak_phase5.csv`, `data/raw/triadmp_phase5_*.csv`).

Interpretation. All four kernels lie left of the ridge, so DRAM-size performance is set by bandwidth; in L1 SAXPY and multiply sit on the L1 bandwidth slope (103% and 102% of the L1 triad line) at 32% and 16% of the FMA peak. The strict dot is neither compute nor memory bound: it is latency bound (3% of peak, 11% of the memory ceiling). The data support a single-thread ceiling of 147 GiB/s of counted traffic and do not support a claim that triad (122 GiB/s) is an upper bound. Write-allocate is not shown to be the reason triad is lower: the out-of-place SAXPY gap of 15% to 17% is smaller than a full extra line stream (25%) would give, and the adjusted copy and scale rates are far above in-place SAXPY.

Failure or anomaly. (1) Read-only (one stream) is the slowest pattern, 80 GiB/s in DRAM against 114 to 139 GiB/s for patterns with two or three streams; the cause is unverified (stream count or prefetch depth is a guess; the loop itself is not the limit in DRAM). HB2's ranking was wrong on this point. (2) One fmla.4s accumulator shows 6.2 cycles latency against about 4 with more accumulators; FMA throughput is non-monotonic in the accumulator count. Both unexplained. (3) With 12 processes the per-process rates are bimodal (68 to 76 vs 27 to 32 GiB/s), consistent with two core tiers (unverified).

10. Negative cases and the naive benchmark

Hypothesis. H4, H16, H21, H18: a float reduction without fast-math does not scale with lanes; removing `restrict` costs a runtime check; naive benchmarks give unphysical numbers.

Result. (A) Reduction without fast-math: vectorized multiply, serial sum, 1.46x over unrolled scalar, flat over working set; fast-math is 10x to 14x faster. The in-order f32 sum has error 3.3, 86, 139, 1.3e5 and 1.1e7 eps at N = 1e3, 1e5, 1e6, 1e7, 1e8 against a double-double reference (additions of terms near 1 are absorbed once the sum exceeds 2^24); fast-math gives 0.5, 1.8, 2.0, 9.4 and 3.8e3 eps. The faster build is also the more accurate one here, and differs from the strict result by up to 1e-2 relative at 1e7. `scalar` and `auto` strict results differ slightly (139 vs 137 eps at 1e6) because scalar uses fused `fmadd`. (B) DRAM-bound SIMD: SAXPY f32 at 1 GiB is perfectly vectorized (12 instructions per 16 elements) and only 1.35x faster than unrolled scalar. (C) Aliasing: without `restrict` the compiler adds a runtime overlap check costing under 7% at large N, 2% at N = 8 to 1024 and 43% at N = 4; with y = x + 1 the check fails and the scalar path runs at 9.4 to 12.8 cycles per element (28x to 67x slower than 0.19 to 0.34). Results stay correct.

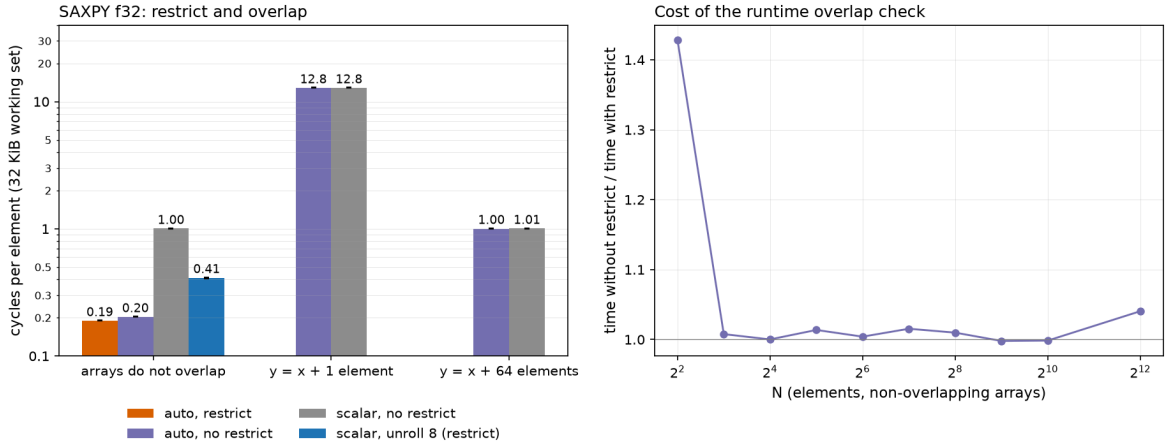


Figure 13. Cost of dropping `restrict`: cycles per element at 32 KiB for non-overlapping and overlapping arguments (left, values on bars) and the small-N cost of the runtime check (right); look for the 12.8 cycles per element when y = x + 1 (`plots/exp7_alias_restrict.png`, raw `data/raw/alias_phase5*.csv`, `data/raw/restrict_smalln_phase5.csv`).

Naive benchmark (an intentionally flawed version written first, with a corrected twin; both datasets kept): N1, a multiply loop with no consumer, timed 0 ns (the timed region contains no loop in the assembly) and reports infinite GiB/s, against 263 GiB/s corrected. N3, first touch inside the timed region, gives 81 against 148 GiB/s (1.8x too low). N2, zero-page reads of a calloc array, did not reproduce (83 against 83 GiB/s); the naive variant had a warm-up pass, so this does not prove the artifact cannot occur. The first corrected N2 still measured 0 ns: a plain memory-clobber barrier does not stop hoisting of calls on memory that never escaped (the pointer must be passed to the asm); that dataset is kept.

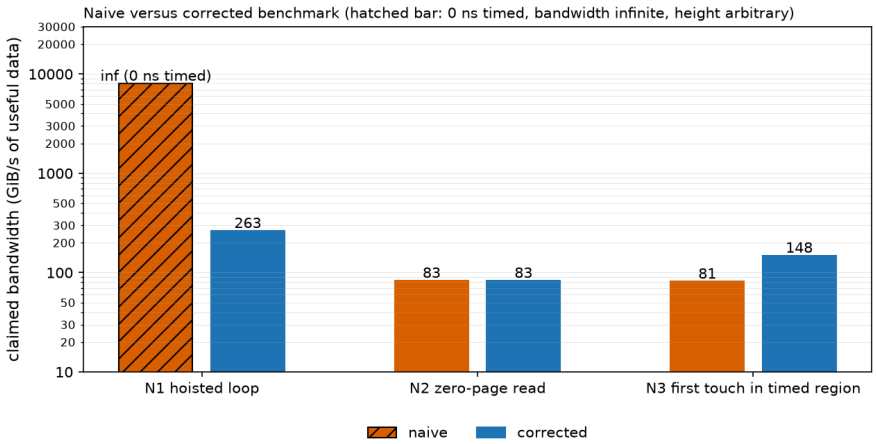


Figure 14. Bandwidth claimed by the naive benchmarks and by their corrected versions (log scale); the hatched N1 bar is infinite (0 ns timed), its height arbitrary (`plots/exp8_naive_vs_fixed.png`, raw `data/raw/naive_phase5.csv`, `data/raw/naive_phase5_run1_fix_still_hoisted.csv`, `vecreports/naive/n1_naive_timed_region.txt`).

Interpretation. The strict reduction is latency bound, not constrained by lanes; the flaws in the naive benchmarks were each identified from the assembly or from implausible numbers. Four further flaws in the harness itself were found and fixed during the work: `restrict` on locals, a non-unrolled default scalar baseline, a 64-bit `long double`, and a later `-Rpass=` option overriding the earlier one (the first remark files were empty).

Failure or anomaly. N2 did not reproduce; the overlap slowdown is above the predicted 4 to 6 cycles per element (cause not tested).

11. Limitations and unexplained results

- No pinning. Placement is by QoS only; four powermetrics samples during a SAXPY run show the Super cluster ran it and the thread migrated among the six Super cores (about 10 to 35% residency per core); the effect of migration was not isolated. Only one benchmark was sampled; placement and clock during the other runs are assumed to match, not measured.
- No hardware counters. `perf` does not exist on macOS; `xctrace` produced aggregate `cycle` and bottleneck metrics whose cycle totals disagreed with timing by about 4.7x and were not used. Claims about vector instructions, FMA pipes and cache misses rest on disassembly and timing.
- The clock is estimated (1-cycle dependent add) and cycles per element assume it is constant; it varies by about 1%, and the final session estimate (4.55 GHz) is 0.8% below the earlier ones.
- Sessions were short (about 10 to 15 minutes), so sustained-throttle behaviour is untested. The load average was 1.2 to 2.4 in the earlier sessions and 2.8 to 3.4 in the final one (a video player was running).
- Unified memory, 16 KiB pages and 128 B lines are Apple specific; the alignment penalty, the flat bandwidth and the stride-32 spike may not generalize to x86 (no x86 host was available).
- Unexplained: the 1 MiB latency step and the 2.0 ns step near 181 pages; the dip at 16 to 32 KiB in the SAXPY curve; the stride-32 spike; the mechanism of the alignment penalty and why a misaligned x alone costs little; the slowest read-only pattern; single-accumulator `fmla.4s` latency and non-monotonic FMA throughput; the in-core cause of the 147 GiB/s plateau.

12. AI-use note

- Tool: Claude Code (Anthropic's CLI agent), working in this repository from task descriptions written by the student; it also executed all builds and benchmarks on the student's machine.
- Generated by the tool: harness, kernel template, correctness tests, FMA-peak and accuracy tools, plotting and analysis scripts, vectorization reports, experiment designs, hypothesis texts with their numeric predictions, the naive-benchmark demonstration, and the first drafts of the report and docs.
- Found and corrected during the work, as recorded in `docs/AI_USE.md`: `restrict` on local pointers, a non-unrolled default scalar baseline, a 64-bit `long double` (replaced by a double-double reference), a `-Rpass=` option that overrode the earlier one, a first corrected naive benchmark that still timed 0 ns, and register-level stencil annotations that had not been traced (removed). The clock estimate was cross-checked with a multiply chain and with powermetrics; validation tolerances were checked by mutation.
- Student contributions recorded: ran powermetrics (three samples, used in Section 3), re-ran the benchmarks, and spot-checked report numbers against the data.
- Not recorded and therefore not claimed: any code or testbench written by team members outside this repository, and any hand modification of the generated code.
- Details, the list of flaws and the verification log are in `docs/AI_USE.md`.

13. Reproducibility

`scripts/run_all.sh` rebuilds, runs the tests, regenerates the vectorization reports and every plot from the committed raw data. `COLLECT=quick scripts/run_all.sh` re-measures a reduced version of every experiment in a fresh clone (result in `docs/REPRODUCTION.md`); `COLLECT=full` repeats the full measurement (about 1 hour). `scripts/make_report_pdf.py` rebuilds the PDF (needs `pip install markdown` and Google Chrome). Raw data: one CSV per run in `data/raw/` with timestamp, session, build, kernel, dtype, N, offsets, stride, repetition, time and thermal state. Session names (`phase3`, `phase4`, `phase5`, `ext1`) are run labels.

14. Evidence index

Plots are in `plots/` (PNG and PDF), processed tables in `data/processed/`, raw CSVs in `data/raw/`, compiler evidence in `vecreports/`.

claim or figure	plot	processed data	raw data	vecreports file
Speedup vs scalar, Table 2, Figures 1, 2 (data types: not embedded)	`exp1_sweep_f32`, `exp1_sweep_f64`, `exp5_dtype`, `exp5_elements_per_ns`	`exp1_key_table_phase3.csv`, `exp1_speedup_phase3.csv`	`sweep_phase3_{A,B,C,D}.csv`	`summary.md`, `asm/scalar_saxpy_f32.s`
Locality plateau, Figure 3	`exp2_saxpy_locality`	`exp1_speedup_phase3.csv`	`sweep_phase3_D.csv`	`asm/auto_saxpy_f32.s`
Pointer chase, Table 3, Figure 4	`exp9_chase_latency`	`extra_chase_ext1.csv`, `extra_chasepage_ext1.csv`, `extra_summary_ext1.txt`	`chase_ext1.csv`, `chasepage_ext1.csv`	none
Vectorization evidence, Section 4	none	none	none	`VECTORIZATION.md`, `remarks/auto_saxpy_f32.txt`, `remarks/auto_saxpy_s2_f32.txt`, `asm/auto_saxpy_f32.s`
Dot decomposition, Table 4, Figure 5	`exp11_dot_decomposition`	`extra_dot_decomposition_ext1.csv`	`dotdec_ext1.csv`	`asm/auto_dot_f32.s`
Relative misalignment, Table 5, Figure 6	`exp12_relative_misalignment`, `exp3_alignment`	`extra_misalign_ext1.csv`, `phase4_summary_phase4.txt`	`misalign_ext1.csv`, `align_phase4.csv`	none
Tail, Figure 7	`exp3_tail_small_n`, `exp3_tail_mod16`	`phase4_summary_phase4.txt`	`tail_phase4.csv`	`asm/auto_saxpy_f32.s`
Stride, gather, Table 6, Figures 8, 9	`exp4_stride_gather`, `exp13_stride_jump`	`phase4_summary_phase4.txt`, `extra_summary_ext1.txt`	`stride_phase4.csv`, `stridejump_ext1.csv`	`remarks/auto_saxpy_s2_f32.txt`, `asm/auto_forcevec_saxpy_s2_f32.s`
Traffic accounting, Table 7, Figure 10	`exp10_traffic_accounting`	`extra_traffic_ext1.csv`	`ceiling_ext1.csv`	none
Roofline, Table 8, Figures 11, 12	`exp6_roofline`, `exp6_peaks`, `exp6_fma_peak`, `exp6_bandwidth_scaling`	`roofline_points_phase5.csv`, `phase5_summary_phase5.txt`	`fmapeak_phase5.csv`, `triad_phase5.csv`, `triadmp_phase5_*.csv`	none
Reduction accuracy (not embedded)	`exp7_reduction_accuracy`	`phase5_summary_phase5.txt`	`accuracy_phase5.csv`	`VECTORIZATION.md`
Aliasing, restrict, Figure 13	`exp7_alias_restrict`	`phase5_summary_phase5.txt`	`alias_phase5*.csv`, `restrict_smalln_phase5.csv`	`asm/auto_norestrict_saxpy_f32.s`
Naive benchmark, Figure 14	`exp8_naive_vs_fixed`	`phase5_summary_phase5.txt`	`naive_phase5.csv`, `naive_phase5_run1_fix_still_hoisted.csv`	`naive/n1_naive_timed_region.txt`
Clock, placement, Section 3	none	`docs/METHODOLOGY.md`	`freq_phase3.csv`, `freq_ext1.csv`, `powermetrics_*_20261006.txt`	none
Hypotheses, Section 2	none	`docs/HYPOTHESES.md`	none	none